

Aula 9 — Sistema de Interrupções

Dois vetores, prioridade, latência e variáveis compartilhadas

Microcontroladores — DENE/UFMT

Objetivos da aula

- Contrapor varredura e evento como modelos de atendimento, e quantificar o custo de cada um.
- Descrever o caminho completo de uma interrupção: condição, sinalizador, habilitação individual, habilitação global e vetor.
- Configurar o sistema de dois níveis de prioridade do PIC18F4550, com seus dois vetores, e reconhecer quando o modo de compatibilidade é preferível.
- Estimar latência e flutuação de atendimento, identificando o que as determina no pior caso.
- Escrever tratadores corretos: salvamento de contexto, limpeza de sinalizadores, uso de `volatile` e proteção de acessos não atômicos.
- Aplicar o padrão arquitetural em que o tratador sinaliza e o laço principal executa.

1 Varredura e evento

Até aqui, todo o firmware do laboratório funcionou por varredura: o laço principal pergunta repetidamente se algo aconteceu — se o botão foi pressionado, se o temporizador estourou, se chegou um caractere na serial. O modelo é simples de entender e de depurar, e é a razão de ele vir primeiro.

Ele tem dois problemas, e ambos são estruturais.

O primeiro é **desperdício**: a maior parte das perguntas recebe “não” como resposta. Um botão pressionado três vezes por minuto, verificado a cada milissegundo, produz dezenas de milhares de leituras inúteis para cada leitura útil.

O segundo é mais sério: **o tempo de resposta depende do resto do programa**. Se em algum ponto do laço há uma escrita no display que leva 2 ms, então qualquer evento que ocorra durante essa escrita espera até 2 ms para ser notado. O atraso não é constante — varia conforme o caminho percorrido no laço — e cresce toda vez que se acrescenta código. É por isso que sistemas construídos apenas por varredura degradam à medida que o projeto avança: cada funcionalidade nova piora o tempo de resposta de todas as anteriores.

Definição — interrupção

Mecanismo pelo qual um evento de hardware suspende a execução do programa corrente, desvia o processador para uma rotina de tratamento e, ao término desta, restaura a execução exatamente no ponto onde foi interrompida — sem que o programa interrompido precise saber que isso ocorreu.

A última cláusula é a essencial. O programa principal não coopera, não é consultado e não precisa ser escrito de forma alguma especial. O desvio é imposto pelo hardware.

2 Anatomia de uma interrupção

Entre o evento físico e a execução do tratador existem três portas em série. Uma interrupção só ocorre se as três estiverem abertas.

Porta	Bit típico	Comportamento
Sinalizador	TMR0IF, RCIF, INT0IF	Ligado pelo hardware quando a condição ocorre. Liga-se mesmo com a interrupção desabilitada. Só é desligado por software — salvo poucas exceções
Habilitação individual	TMR0IE, RCIE, INT0IE	Autoriza aquela fonte específica a solicitar o desvio
Habilitação global	GIE, PEIE	Autoriza o conjunto. Desligá-la suspende o atendimento sem perder os eventos

Tabela 1: As três portas em série no caminho de uma interrupção.

Observação

A independência entre a primeira e as demais é o que torna a varredura por sinalizador possível: pode-se deixar **TMR0IE** desligado e simplesmente testar **TMR0IF** no laço principal. O sinalizador continua sendo ligado pelo hardware. Muitas vezes essa é a solução certa — interrupção não é obrigatória para usar um periférico.

Aberto o caminho, o hardware executa a sequência: conclui a instrução em curso, empilha o endereço de retorno, desliga a habilitação global para evitar reentrada e desvia para um endereço fixo, o **vetor de interrupção**.

3 Os dois vetores do PIC18

Aqui está a diferença mais relevante em relação às famílias de 8 bits mais antigas, e a que mais exige atenção de quem vem do PIC16.

Endereço	Papel	Observação
0x0000	Vetor de reinicialização	Primeira instrução após qualquer reset
0x0008	Vetor de alta prioridade	Vetor único quando IPEN = 0
0x0018	Vetor de baixa prioridade	Existe apenas quando IPEN = 1

Tabela 2: Vetores do PIC18F4550.

O bit **IPEN**, no registrador **RCON**, escolhe entre dois regimes de operação:

Modo de compatibilidade (**IPEN = 0**). Todas as fontes desviam para **0x0008**. Não há prioridade: quem chegar primeiro é atendido, e uma interrupção em atendimento não pode ser interrompida por outra. É o comportamento das famílias anteriores, e é o regime adequado quando todas as fontes têm exigências temporais semelhantes.

Modo de prioridade (**IPEN = 1**). Cada fonte é classificada como alta ou baixa prioridade por um bit próprio, e desvia para o vetor correspondente. A regra de precedência é assimétrica

e precisa ser memorizada: **uma fonte de alta prioridade interrompe um tratador de baixa prioridade em execução; o contrário nunca ocorre.**

Atenção

Com $IPEN = 1$, os bits de habilitação global mudam de nome e de significado: **GIE** passa a ser **GIEH**, habilitação global das interrupções de alta prioridade, e **PEIE** passa a ser **GIEL**, das de baixa. Um programa que habilita apenas **GIE** nesse modo, esperando o comportamento antigo, deixa todas as fontes de baixa prioridade permanentemente sem atendimento — e o defeito se manifesta como “essa interrupção não funciona”, sem qualquer erro de compilação.

3.1 Onde ficam os bits

O sistema é distribuído por vários registradores, o que é a principal fonte de confusão inicial. A organização, porém, é regular:

Grupo de fontes	Sinalizador	Habilitação	Prioridade
Temporizador 0, INT0, mudança em RB	INTCON	INTCON	INTCON2
INT1 e INT2	INTCON3	INTCON3	INTCON3
Periféricos grupo 1	PIR1	PIE1	IPR1
Periféricos grupo 2	PIR2	PIE2	IPR2

Tabela 3: Organização dos registradores de interrupção.

A regra mnemônica é direta: para os periféricos, os três registradores diferem por uma letra — **PIR** para o sinalizador, **PIE** para a habilitação, **IPR** para a prioridade — e o bit ocupa a mesma posição nos três.

Atenção

A interrupção externa INT0 não tem bit de prioridade. Ela é sempre de alta prioridade quando $IPEN = 1$. Procurar por **INT0IP** na folha de dados é um exercício frustrante: ele não existe. INT1 e INT2 têm o seu; INT0, não.

4 Latência

Definição — latência de interrupção

Intervalo entre o instante em que a condição de interrupção ocorre e o instante em que a primeira instrução do tratador é executada.

No PIC18, a parte devida ao hardware é pequena e bem definida: cerca de três a quatro ciclos de instrução, correspondentes ao término da instrução corrente, ao empilhamento e ao desvio.

Exemplo

Com o oscilador em 48 MHz, um ciclo de instrução dura $T_{cy} = \frac{4}{48} \text{ MHz} \approx 83 \text{ ns}$. Quatro ciclos correspondem a cerca de 330 ns. Comparado ao pior caso de uma varredura num laço que contenha uma escrita de display de 2 ms, a diferença é de aproximadamente **seis mil vezes**.

Essa conta, porém, é a parte fácil e a menos interessante. A latência real do sistema é determinada por outras três parcelas, todas sob responsabilidade do programador:

1. **Trechos com interrupções desabilitadas.** Toda seção crítica adia o atendimento de tudo.
2. **Tratadores longos.** Enquanto um tratador de alta prioridade executa, nenhuma outra fonte é atendida. Um tratador de 500 μs impõe essa latência a todo o resto do sistema.
3. **Salvamento de contexto**, quando ele não é automático — assunto da próxima seção.

Observação

Daí decorre a regra prática mais importante desta aula, e ela é arquitetural, não estilística: **o tratador deve ser o mais curto possível**. Não porque tratadores longos sejam deselegantes, mas porque o mais lento deles define a latência de todo o sistema.

A **flutuação**, ou incerteza da latência entre uma ocorrência e outra, importa tanto quanto o valor médio. Um sistema com latência de 300 ns e flutuação de 500 μs é, para efeitos de temporização precisa, um sistema de 500 μs .

5 Salvamento de contexto

O tratador executa entre duas instruções quaisquer do programa principal e usa os mesmos registradores que ele. Se alterar o acumulador ou os sinalizadores de estado sem restaurá-los, o programa interrompido retomará com valores corrompidos — e o defeito resultante é intermitente e dependente do instante exato da interrupção, isto é, praticamente irreproduzível em depuração.

O PIC18 oferece uma solução em hardware: existem **registradores sombra** que guardam automaticamente o acumulador, o registrador de estado e o de seleção de banco na entrada da interrupção, com restauração automática no retorno rápido.

Atenção

Esse recurso automático serve à interrupção de **alta** prioridade. O tratador de baixa prioridade pode ter seu contexto sobrescrito por uma interrupção de alta que o interrompa, de modo que seu contexto precisa ser preservado em memória. Na prática, o compilador XC8 gera esse código; o que cabe ao programador é saber que ele existe, que custa ciclos e que o custo aparece na latência do sistema.

Código

```
/* Dois tratadores distintos, um por vetor. */  
  
void __interrupt(high_priority) isr_alta(void)  
{
```

```
    if (INTCONbits.TMR0IF && INTCONbits.TMR0IE) {
        INTCONbits.TMR0IF = 0;      /* limpar SEMPRE */
        TMR0H = PRECARGA_H;
        TMR0L = PRECARGA_L;
        g_tique = 1;                 /* apenas sinaliza */
    }
}

void __interrupt(low_priority) isr_baixa(void)
{
    if (PIR1bits.RCIF && PIE1bits.RCIE) {
        rx_buffer[rx_fim] = RCREG; /* a leitura limpa RCIF */
        rx_fim = (rx_fim + 1u) % RX_TAM;
    }
}
```

Duas convenções aparecem nesse trecho e valem para todo tratador. A primeira: o teste combina o sinalizador e a habilitação. Sem isso, uma fonte desabilitada cujo sinalizador esteja ligado seria tratada indevidamente. A segunda: os testes são encadeados em ordem de urgência, porque um único vetor atende a várias fontes — é o tratador que descobre quem o chamou.

6 Sinalizadores: as regras de limpeza

Atenção

Um sinalizador não limpo faz o tratador ser chamado indefinidamente. Ao retornar, a condição continua sinalizada, e o hardware desvia de novo. O sintoma é o programa principal aparentemente travado, sem nunca progredir — quando, na verdade, ele está sendo interrompido continuamente.

Três casos merecem atenção especial, porque a limpeza direta não basta:

Recepção serial. RCIF não é limpo por escrita: ele se apaga quando o registrador de recepção é lido. Se o tratador não lê o dado recebido, a interrupção se repete.

Mudança de estado em RB. O sinalizador correspondente só pode ser baixado após a leitura da porta B, que é o que encerra a condição de discrepância interna. A sequência correta é ler a porta, depois limpar o sinalizador — nunca o inverso.

Comparadores. Como visto na Aula 5, a leitura de CMCON precede a limpeza de CMIF.

Observação

Há um padrão comum: nesses três casos o hardware exige que o software **reconheça o dado** antes de aceitar a limpeza. Isso não é capricho — é uma proteção contra perder eventos, garantindo que a informação foi de fato consumida.

7 Variáveis compartilhadas

Esta seção trata da classe de defeitos mais difícil de depurar em sistemas embarcados, e a razão é sempre a mesma: o tratador e o programa principal acessam a mesma variável, mas o programador raciocina como se um só existisse.

7.1 A palavra-chave `volatile`

O compilador otimiza sob a premissa de que só o código visível altera a memória. Ao ver um laço que testa uma variável nunca modificada dentro dele, ele conclui — corretamente, do seu ponto de vista — que o valor não muda, lê a variável uma única vez e a mantém num registrador.

Código

```
volatile uint8_t g_tique = 0;    /* alterada dentro do tratador */

/* Sem 'volatile', o laço abaixo pode ser compilado como
   um laço infinito: o compilador não vê ninguém alterando
   g_tique e otimiza a releitura. */
while (!g_tique) { }
```

Definição — `volatile`

Qualificador que informa ao compilador que a variável pode ser alterada por algo fora do fluxo visível de execução — um tratador de interrupção ou o próprio hardware. Cada acesso no código-fonte deve produzir um acesso real à memória, sem eliminação nem reordenação.

A regra de aplicação é objetiva: **toda variável escrita no tratador e lida fora dele, ou o inverso, precisa ser `volatile`**. Não usá-la produz um programa que funciona sem otimização e falha quando ela é ligada — um dos relatos mais frequentes de “o compilador está errado”.

7.2 Acesso não atômico

O segundo problema é mais sutil e não se resolve com `volatile`.

Derivação

Considere um contador de 16 bits, incrementado a cada milissegundo pelo tratador, e lido pelo laço principal. Num processador de 8 bits, a leitura são duas instruções: primeiro o byte baixo, depois o alto.

Suponha o contador em `0x00FF`, prestes a passar para `0x0100`. A sequência pode ser:

1. o laço principal lê o byte baixo: `0xFF`;
2. **ocorre a interrupção**; o contador passa a `0x0100`;
3. o laço principal lê o byte alto: `0x01`.

O valor montado é `0x01FF` — 511 — quando os valores corretos seriam 255 antes ou 256 depois. O erro tem 256 unidades de magnitude, é raro, e depende de um alinhamento temporal específico entre a leitura e a interrupção.

Atenção

Esse defeito ocorre em intervalos que podem ser de horas e desaparece quando se acrescenta um ponto de parada para observá-lo. Se um sistema apresenta falhas raras, inexplicáveis e não reproduzíveis, **variáveis multibyte compartilhadas são o primeiro lugar a investigar**.

Há duas soluções. A primeira é a **seção crítica**: desabilitar a interrupção durante a leitura.

Código

```
uint16_t ler_contador(void)
{
    uint16_t v;

    INTCONbits.GIEH = 0;          /* seção crítica: início */
    v = g_contador;
    INTCONbits.GIEH = 1;          /* fim */

    return v;
}
```

Ela é correta, e tem um preço: aumenta a latência de todas as interrupções pelo tempo em que a porta permanece fechada. Seções críticas devem ser as menores possíveis — e nunca conter espera, escrita em display ou chamada de função de biblioteca.

A segunda solução é a **leitura dupla**: ler, ler de novo e repetir enquanto os dois valores diferirem. Não fecha a porta e é preferível quando a leitura é frequente e a interrupção, rara.

7.3 Reentrância

Atenção

Chamar a mesma função a partir do tratador e do laço principal é um erro clássico nesta família. O compilador XC8 aloca as variáveis locais em endereços fixos, e não em uma pilha — o que torna as funções não reentrantes por padrão. Se a interrupção ocorrer no meio da execução da função pelo programa principal, as duas invocações compartilharão as mesmas posições de memória, e ambas produzirão resultados errados. O compilador sinaliza o conflito quando consegue detectá-lo; não convém depender disso.

8 O padrão arquitetural: sinalizar e executar

Reunindo as restrições — tratador curto, seções críticas mínimas, sem chamadas compartilhadas —, resta um padrão de organização que atravessa todo o resto do curso e reaparece em qualquer plataforma.

Observação

O tratador não executa a tarefa. Ele registra que a tarefa deve ser executada. O trabalho ocorre no laço principal, fora do contexto de interrupção, onde a duração não compromete a latência de mais ninguém.

Código

```

/* Tratador: minimo. Registra a passagem do tempo. */
void __interrupt(high_priority) isr_alta(void)
{
    if (INTCONbits.TMR0IF) {
        INTCONbits.TMR0IF = 0;
        TMR0H = PRECARGA_H;
        TMR0L = PRECARGA_L;
        g_tique = 1;
    }
}

/* Laco principal: executa, sem pressa e sem bloquear ninguem. */
for (;;) {
    if (!g_tique) {
        continue;
    }
    g_tique = 0;

    contador_lcd += TICK_MS;
    if (contador_lcd >= 3000u) {      /* tarefa lenta, fora da ISR */
        contador_lcd = 0;
        mostrar_lcd();
    }
}

```

A escrita no display leva milissegundos — e não custa latência a ninguém, porque acontece fora do tratador. É o mesmo escalonador cooperativo do encontro 7, agora com a base de tempo vinda de interrupção em vez de espera ativa. O sistema passa a ter duas camadas com responsabilidades distintas: o hardware marca **quando**; o laço principal decide **o quê**.

9 Defeitos característicos

Sintoma	Causa provável	Verificação
Programa principal nunca progride	Sinalizador não limpo no tratador	Conferir a limpeza e as regras especiais de RB, serial e comparador
A interrupção nunca ocorre	Habilitação global incompleta com <code>IPEN = 1</code>	Conferir <code>GIEH</code> e <code>GIEL</code> , e o bit de prioridade da fonte
Funciona sem otimização, falha com ela	Falta <code>volatile</code>	Listar as variáveis compartilhadas com o tratador
Falha rara, não reproduzível	Acesso não atômico a variável multibyte	Proteger a leitura ou usar leitura dupla
Valores corrompidos no laço principal	Função não reentrante chamada dos dois contextos	Verificar o grafo de chamadas do tratador

Sintoma	Causa provável	Verificação
Temporização com flutuação	Tratador longo ou seção crítica extensa	Medir com um pino de depuração e osciloscópio

Tabela 4: Defeitos característicos e como investigá-los.

Observação

A última linha sugere a técnica de depuração mais útil desta aula: **ligar um pino no início do tratador e desligá-lo no fim**, observando o resultado no osciloscópio. A largura do pulso é a duração do tratador; o intervalo entre pulsos é a taxa de ocorrência; e a variação da borda de subida em relação ao evento é a flutuação. Três medidas que nenhum depurador passo a passo fornece, porque parar o processador destrói exatamente o fenômeno que se quer observar.

10 Transposição para arquiteturas modernas

O modelo conceitual — sinalizador, habilitação, vetor, prioridade, contexto — é o mesmo em qualquer arquitetura. O que muda nas famílias ARM Cortex-M é a escala e o grau de automação, e a comparação é instrutiva:

- **Um vetor por fonte.** Em vez de dois endereços com testes encadeados no tratador, há uma tabela com uma entrada por periférico. O desvio já chega na rotina certa, e desaparece a cadeia de `if` do início do tratador.
- **Prioridades numéricas configuráveis.** Em vez de dois níveis, tipicamente dezenas, com aninhamento gerenciado pelo controlador de interrupções.
- **Empilhamento automático.** O processador salva parte dos registradores automaticamente, o que permite escrever o tratador como uma função C comum, sem qualificador especial.
- **Encadeamento de interrupções.** Interrupções pendentes em sequência dispensam a restauração e o novo empilhamento intermediários, reduzindo o custo do atendimento consecutivo.

Atenção

O que **não** muda: os sinalizadores dos periféricos continuam precisando ser limpos, `volatile` continua obrigatório, seções críticas continuam aumentando latência, e acesso não atômico continua causando falhas raras — agora em variáveis de 32 bits acessadas por sistemas de 32 bits, o que apenas desloca o problema para tipos maiores. Tudo o que esta aula tem de difícil permanece exatamente igual.

11 Exercícios

Exercício 9.1

Um programa configura `IPEN = 1`, define a recepção serial como baixa prioridade e habilita `RCIE` e `GIE`. A interrupção de recepção nunca ocorre, embora os dados cheguem corretamente e possam ser lidos por varredura. Explique a causa e apresente a correção.

Exercício 9.2

Com o oscilador em 48 MHz, um tratador de alta prioridade executa em 40 ciclos de instrução, incluindo entrada e saída. Uma fonte de baixa prioridade ocorre no instante em que esse tratador acaba de iniciar. Calcule a latência da fonte de baixa prioridade e discuta o que aconteceria se ela ocorresse a cada 5 μ s.

Exercício 9.3

Escreva a função `ler_contador` da seção 7 usando a técnica de leitura dupla, sem desabilitar interrupções. Explique em que condição essa versão é preferível à seção crítica e em que condição não é.

Exercício 9.4

O tratador abaixo pretende contar pulsos e acender um aviso a cada cem. Aponte **três** defeitos distintos e reescreva-o.

```
uint16_t pulsos = 0;

void __interrupt(high_priority) isr(void)
{
    pulsos++;
    if (pulsos % 100 == 0) {
        lcd_cursor(1, 0);
        lcd_texto("100 pulsos");
    }
}
```

Exercício 9.5

Argumente a favor da seguinte afirmação e depois contra ela, com pelo menos dois argumentos técnicos de cada lado: “num sistema com uma única fonte de interrupção, o modo de compatibilidade (`IPEN = 0`) é sempre preferível ao modo de prioridade”.